

Plan SQA - StockSync

1. Datos del proyecto

CAMPO	Valor
Nombre del proyecto	StockSync
Grupo N°	5
Integrantes	• Alasino Benjamin • Calvo Tomas Sebastian • Contreras Joaquin • Gonzalez Martin
Referente del grupo	Alasino Benjamin
Fecha del plan	20/05/2026
Versión del plan	v1.0.0
Repositorio	https://github.com/benjaalasino/StockSync

2. Propósito del plan

El presente Plan SQA tiene como objetivo garantizar que el sistema StockSync cumpla con los estándares de calidad establecidos a lo largo de su desarrollo, asegurando que el software sea funcional, mantenible y verificable. Se busca mitigar los principales riesgos del proyecto: la generación de stock negativo por condiciones de carrera en ventas concurrentes, la pérdida de trazabilidad en los movimientos de inventario, y la introducción de defectos por falta de revisión sistemática del código. Para ello, el plan define métricas objetivas de calidad, establece roles y responsabilidades dentro del equipo, y fija una estrategia de revisiones, pruebas automatizadas y análisis estático que acompañe cada etapa del desarrollo.

3. Roles del equipo

Rol	Persona
Responsable de QA (procesos)	Contreras Joaquin
Responsable de QC (testing)	Gonzalez Martin
Documentador	Calvo Tomas
Referente con el docente	Alasino Benjamin

4. Estándares y convenciones

Aspecto	Convención adoptada
Estilo de código	PEP 8 (Python)
Convención de naming	snake_case

Mensajes de commit	Convencional (feat: , fix:)
Encoding de archivos	UTF-8
Indentación	Tab
Nombres de archivos	snake_case

Documentación inline

- [] Cada función pública tiene docstring
- [] Las clases tienen docstring con propósito
- [] Los módulos tienen comentario inicial con su responsabilidad
- [] Los TODOs tienen autor y fecha (# TODO(juan, 2026-05-12): ...)

5. Estrategia de revisiones

Pregunta	Respuesta
¿Es obligatorio code review antes de merge a main ?	Sí
¿Cuántas aprobaciones se requieren?	2
¿Quién puede aprobar?	Copilot - Integrante (no autor)
¿Hay ramas protegidas?	Sí (main bloqueada)

Checklist de revisión (antes de aprobar un PR)

- [] El código compila / corre sin errores
- [] El linter no reporta errores críticos
- [] Los tests pasan
- [] Hay tests para el nuevo código
- [] Los nombres de variables y funciones son claros
- [] No hay código comentado sin razón
- [] No hay secretos hardcodeados (passwords, API keys)
- [] La trazabilidad (Trazabilidad: REQ-XXX) está actualizada en docstrings

6. Métricas a monitorear y umbrales

Métrica	Umbral del proyecto	Cómo se mide
Complejidad ciclomática máxima por función	≤ 10	radon cc
Maintainability Index mínimo por módulo	≥ 40	radon mi
Cobertura mínima de tests	$\geq 60\%$	pytest --cov
Errores críticos del linter	0	ruff check
Defectos críticos abiertos	0	Issues GitHub

al cierre		
	Métrica	Al cierre
	Complejidad ciclomática máxima por función	≤ 10
	Maintainability Index mínimo por módulo	≥ 40
	Cobertura mínima de tests	$\geq 60\%$
	Errores críticos del linter	0
	Defectos críticos abiertos	0

7. Herramientas

Función	Herramienta elegida	Justificación
Linter	Ruff (Python) / ESLint (TypeScript)	Ruff reemplaza flake8 + isort en un solo tool, es extremadamente rápido y tiene reglas configurables. ESLint es el estándar de facto para TypeScript.
Framework de tests	pytest (Python) / Vitest (TypeScript)	Pytest es el framework más adoptado en el ecosistema Python, con soporte nativo de fixtures y parametrización. Vitest se integra nativamente con Vite sin configuración adicional.
Cobertura	pytest-cov + coverage.py	Se integra directamente con pytest mediante un flag (--cov), genera reportes en HTML y XML, y permite definir umbrales mínimos de cobertura que fallan el build si no se alcanzan.
Métricas (CC/MI)	Radon	Herramienta Python específica para calcular Complejidad Ciclomática (radon cc -s) e Índice de Mantenibilidad (radon mi -s) por módulo, que son exactamente las métricas que pide la consigna.
Gestión de defectos	GitHub Issues	Permite registrar defectos

		con severidad (labels), asignar responsables, vincularlos a commits/PRs y hacer seguimiento de estado (abierto/cerrado), todo dentro del mismo repositorio.
Control de versiones	Git + GitHub	Flujo GitFlow con ramas <i>feature</i> /, PRs hacia <i>develop</i> , commits con formato convencional (feat: , fix:)
CI/CD (opcional)	GitHub Actions	Se configura con un archivo .yml en el repo. Puede correr el linter y los tests automáticamente en cada PR, garantizando que ningún merge rompa el build.

8. Frecuencia de medición

Evento	Qué se mide / verifica
Cada commit	Linter (local pre-commit hook si es posible)
Cada Pull Request	Tests + cobertura + linter
Semanal (auto-revisión del grupo)	Métricas radon (CC, MI)
Antes de cada hito	Reporte completo + actualización del Plan SQA

9. Gestión de defectos

- ☐ Todo defecto se carga como Issue en el repositorio
- ☐ Cada issue tiene: título, descripción, pasos para reproducir, severidad
- ☐ Severidades: Crítico / Mayor / Menor / Cosmético
- ☐ Antes de cerrar un issue, se escribe el test que verifica el fix
- ☐ No se mergea código nuevo si hay issues críticos abiertos

Criterios para cerrar un issue:

- ☐ El bug está reproducido y tiene un test que falla antes del fix
- ☐ El fix pasa el test agregado
- ☐ El fix pasa el resto de la suite de tests
- ☐ El fix pasó por code review
- ☐ El issue está documentado con commit asociado

10. Criterios de aceptación para la entrega final (Hito 5)

Criterio	¿Se cumple?
Tests pasan al 100%	☒
Cobertura $\geq$ 60 % sobre módulos clave	☒

Sin issues críticos abiertos	
RTM completa (cada REQ con código + test)	
README con instrucciones de instalación y uso	
Plan SQA actualizado	
Plan de pruebas documentado	
Wireframes en alta y baja fidelidad (Tema VIII)	
Reflexión final de 1 página	

11. Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
Condición de carrera en ventas concurrentes genera stock negativo	Baja	Alto	Uso de SELECT FOR UPDATE en transacciones de venta. Validación atómica antes del commit con rollback + HTTP 409.
Pérdida de trazabilidad en movimientos de inventario	Baja	Alto	Tabla StockMovement append-only. Nunca se hace UPDATE/DELETE. El stock se calcula siempre como SUM(quantity)
Introducción de defectos por falta de revisión	Media	Medio	Code review obligatorio con 2 aprobaciones antes de merge a main. CI con linter + tests en cada PR.
Baja cobertura de pruebas en módulos críticos	Media	Medio	Umbral mínimo de cobertura ($\geq 80\%$) en pyproject.toml. fail_under = 60 con meta de 80% para entrega final.
Dependencias con vulnerabilidades de seguridad	Baja	Alto	Uso de dependencias versionadas. Secretos manejados con variables de entorno, no hardcodeados. JWT con expiración.
Regresión al agregar nuevas funcionalidades	Media	Medio	Suite de 48 tests automatizados que deben pasar antes de
			cada merge. Cobertura del 91.37% brinda red de seguridad.

12. Cronograma de hitos

Fecha	Hito interno del grupo
29/04	Conformación de grupos + idea del proyecto
05/05	Propuesta de proyecto
20/05	HITO 3 — entrega del Plan SQA + métricas iniciales

03/06	HITO 4 — entrega de plan de pruebas + reporte de coberturas
16/06	HITO 5 — entrega final: métricas finales, documentos actualizados, wireframes + mockups, defensa oral